



Лекція 5

Основи розробки web-застосунків

Servlets, JSP

Web-application

Web application is a client-server computer program that the client (including the user interface and client-side logic) runs in a web browser

Wikipedia

Back-end - код, що виконується на серверному боці web-застосунку

Front-end - код, що виконується на клієнтському боці web-застосунку (браузері)

Back-end для виконання своєї функції реалізовує **API**, яке використовує front-end

Передбачається, що вже знаєте

Базові принципи роботи протокола HTTP:

- hostname, port, URL
- request/response, params, headers, methods, response codes

HTML (в базовому вигляді)

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>

Web container

Web container (aka Servlet container) - компонент web-сервера, який взаємодіє з Java-сервлетами

- Керує їх життєвим циклом
- Забезпечує мапінг
- Керує доступом
- Забезпечує перенаправлення запитів

Web container реалізовує **Java Servlet specification**

Servlet API history				
Servlet API version	Released	Specification	Platform	Important Changes
Jakarta Servlet 6.0	May 31, 2022	6.0	Jakarta EE 10	remove deprecated features and implement requested enhancements
Jakarta Servlet 5.0	Oct 9, 2020	5.0	Jakarta EE 9	API moved from package <code>javax.servlet</code> to <code>jakarta.servlet</code>
Jakarta Servlet 4.0.3	Sep 10, 2019	4.0	Jakarta EE 8	Renamed from "Java" trademark
Java Servlet 4.0	Sep 2017	JSR 369	Java EE 8	HTTP/2
Java Servlet 3.1	May 2013	JSR 340	Java EE 7	Non-blocking I/O, HTTP protocol upgrade mechanism (WebSocket) ^[13]
Java Servlet 3.0	December 2009	JSR 315	Java EE 6, Java SE 6	Pluggability, Ease of development, Async Servlet, Security, File Uploading
Java Servlet 2.5	September 2005	JSR 154	Java EE 5, Java SE 5	Requires Java SE 5, supports annotation
Java Servlet 2.4	November 2003	JSR 154	J2EE 1.4, J2SE 1.3	web.xml uses XML Schema
Java Servlet 2.3	August 2001	JSR 53	J2EE 1.3, J2SE 1.2	Addition of <code>Filter</code>
Java Servlet 2.2	August 1999	JSR 902 , JSR 903	J2EE 1.2, J2SE 1.2	Becomes part of J2EE, introduced independent web applications in <code>.war</code> files
Java Servlet 2.1	November 1998	2.1a	Unspecified	First official specification, added <code>RequestDispatcher</code> , <code>ServletContext</code>
Java Servlet 2.0	December 1997	—	JDK 1.1	Part of April 1998 Java Servlet Development Kit 2.0 ^[14]
Java Servlet 1.0	December 1996	—		Part of June 1997 Java Servlet Development Kit (JSDK) 1.0 ^[6]



Apache Tomcat

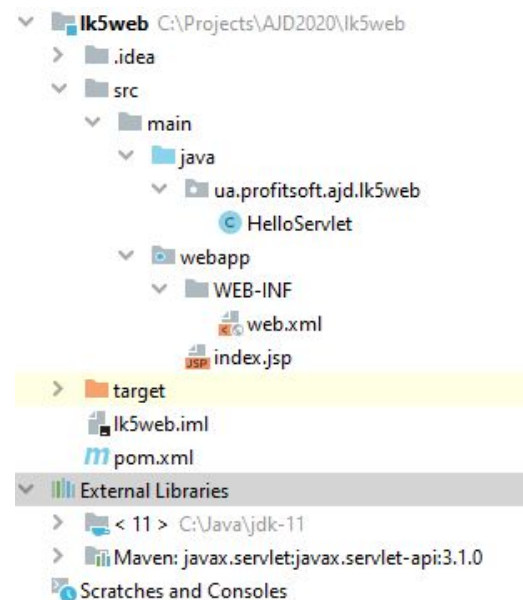
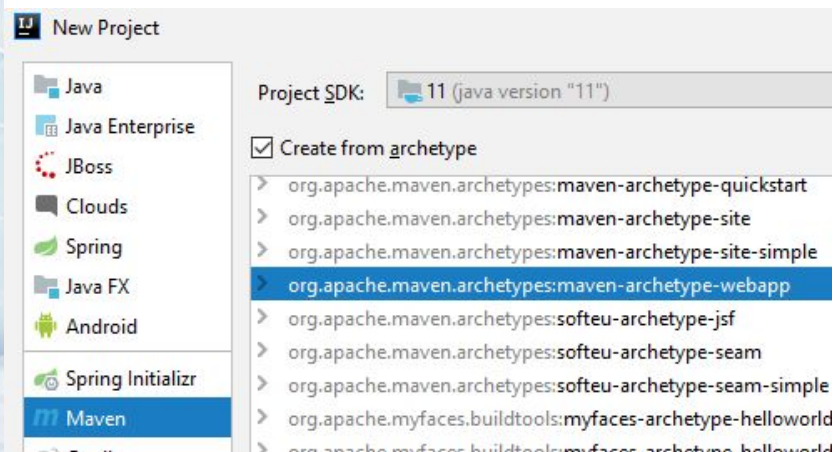
Tomcat - контейнер сервлетів із відкритим вихідним кодом, що розробляється Apache Software Foundation.

Реалізує специфікацію сервлетів, специфікацію JavaServer Pages (JSP) та JavaServer Faces (JSF)

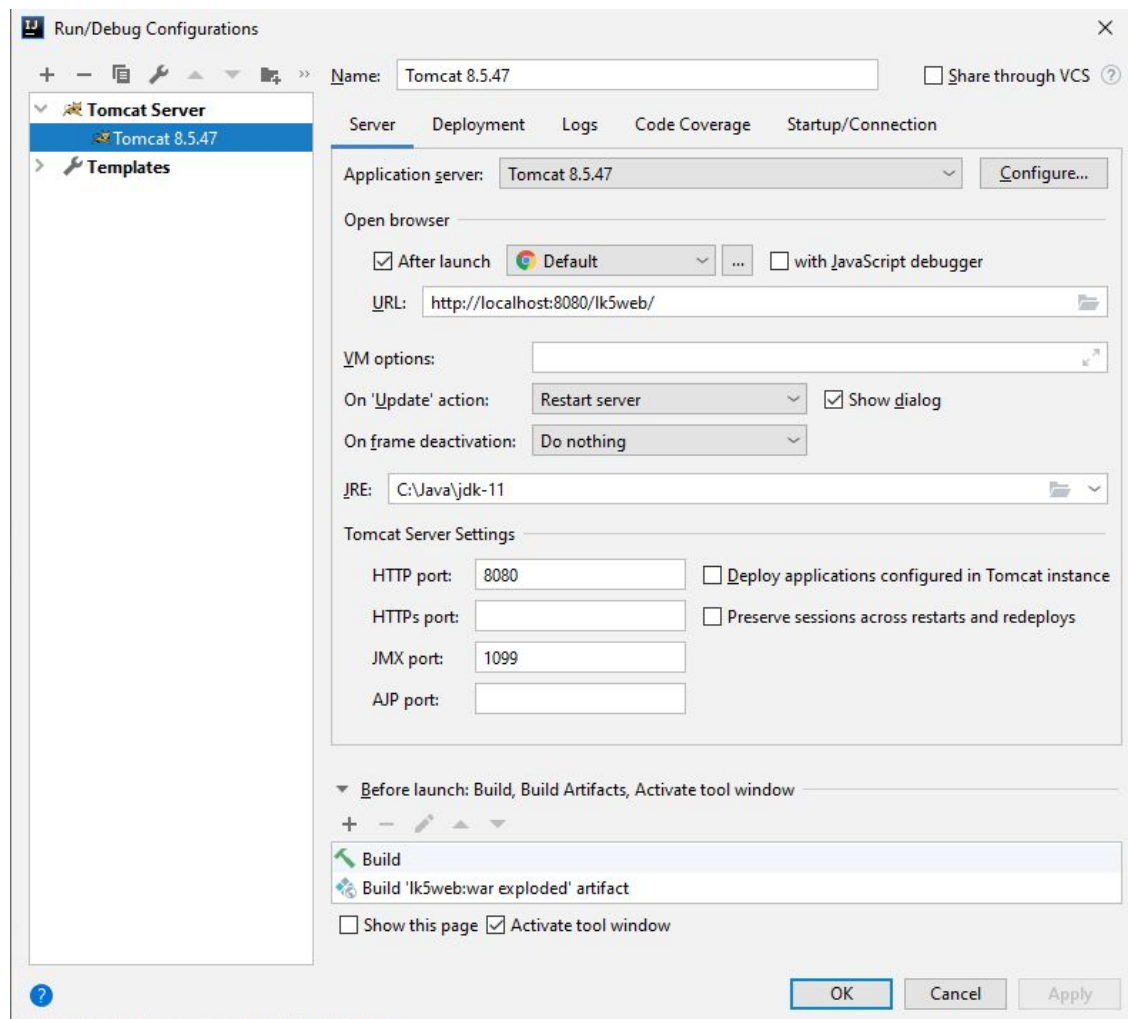
Currently Supported Versions

Servlet Spec	Pages Spec	JSP Spec	EL Spec	WebSocket Spec	Authentication Spec (JASPIC)	Annotation Spec	Apache Tomcat Version	Latest Released Version	Supported Java Versions
6.1	4.0	2.0	6.0	2.2	3.1	3.0	11.0.x	11.0.14	17 and later
6.0	3.1	2.0	5.0	2.1	3.0	2.1	10.1.x	10.1.49	11 and later
4.0	2.3	1.0	3.0	1.1	1.1	1.3	9.0.x	9.0.112	8 and later

Створення і структура web-проекта



Запуск web-проекта



pom.xml updates

```
<properties>  
  <maven.compiler.source>17</maven.compiler.source>  
  <maven.compiler.target>17</maven.compiler.target>  
</properties>
```

...

```
<dependency>  
  <groupId>javax.servlet</groupId>  
  <artifactId>javax.servlet-api</artifactId>  
  <version>3.1.0</version>  
  <scope>provided</scope>  
</dependency>
```


Дескриптор розгортання web.xml

Знаходиться в каталозі **WEB-INF**

В ньому декларується

- Сервлети і мапінг до них
- Слухачі подій
- Фільтри
- Початкові сторінки
- Життєвий час сесії
- Та інше

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
  http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
  version="3.1">
```

...

```
</web-app>
```

Сервлет - це клас, який реалізує інтерфейс `javax.servlet.Servlet`

- Для протокола HTTP зручно наслідувати `javax.servlet.http.HttpServlet`

Функції і можливості сервлетів

- Доступ до заголовків і вмістів запитів (параметри, заголовки, тіло запита)
- Звернення до компонентів бізнес-логіки і отримання результатів
- Перенаправлення запитів (redirect або forward)
- Генерація відповіді (вмістів і заголовків)
 - HTML, XML, JSON, бінарні дані

Життєвий цикл сервлета

Життєвий цикл сервлета **керується контейнером**, в якому сервлет був розгорнутий

При зверненні до Servlet контейнер виконує наступні кроки:

- Якщо екземпляра сервлета не існує, Servlet Container
 - Завантажує клас сервлета
 - Створює екземпляр класу сервлета
 - Ініціалізує екземпляр сервлета, викликаючи метод **init()**
- Викликає метод **service()**, передає йому об'єкти запиту і відповіді

Якщо контейнеру потрібно видалити сервлет, він його фіналізує, викликаючи метод **destroy()**

Клас `HttpServlet`

Використовується для обробки запитів за протоколом **HTTP**

Метод `service()` класа `HttpServlet` викликає один із методів `doXxx()`, в залежності від типу запитів:

- `doGet(HttpServletRequest req, HttpServletResponse resp)` – використовується для обробки GET-запитів
- `doPost(HttpServletRequest req, HttpServletResponse resp)` – використовується для обробки POST-запитів
- `doPut(...)`
- `doDelete(...)`, та інші

Інтерфейс HttpServletRequest

Нащадок `ServletRequest`, зберігає інформацію про запит клієнта і передається у вигляді параметра методам `doXxx()`

Методи:

- `String getParameter(String name)`
 - Параметри запиту
- `get/setAttribute(String)`
 - Атрибути проміжної обробки запита
- `String getHeader(String)`
 - Доступ до заголовків запиту
- `getInputStream()` / `getReader()`
 - Доступ до тіла запиту
- `getRemoteHost()`
- `getRequestURL()`

Інтерфейс HttpServletResponse

Цей інтерфейс є наслідником `ServletResponse` і зберігає інформацію про відповідь клієнту

Методи HttpServletResponse

- `addHeader(String name, String val)`
- `setHeader(String name, String val)`
- `setLocale(Locale loc)`
- `setStatus(int code)`
- `sendError(int code)`
- `setContentLength(int len)`
- `setContentType(String type)`
- `getOutputStream() / getWriter()`
- `sendRedirect(String location)`

Приклад сервлета

```
public class HelloServlet extends HttpServlet {  
    @Override  
    protected void doGet(HttpServletRequest req,  
        HttpServletResponse resp) throws IOException {  
        String nameParam = req.getParameter("name");  
        String name = nameParam != null && !nameParam.isBlank() ?  
            nameParam : "Stranger";  
        PrintWriter writer = resp.getWriter();  
        writer.print("Hello, ");  
        writer.print(name);  
        writer.print("!");  
        writer.flush();  
    }  
}
```


Мабінг сервлета (в web.xml)

```
<servlet>
```

```
  <servlet-name>Hello</servlet-name>
```

```
  <servlet-class>
```

```
    dev.profitsoft.fsd.lk5web.HelloServlet
```

```
  </servlet-class>
```

```
</servlet>
```

```
<servlet-mapping>
```

```
  <servlet-name>Hello</servlet-name>
```

```
  <url-pattern>/hello</url-pattern>
```

```
</servlet-mapping>
```

Аннотація @WebServlet

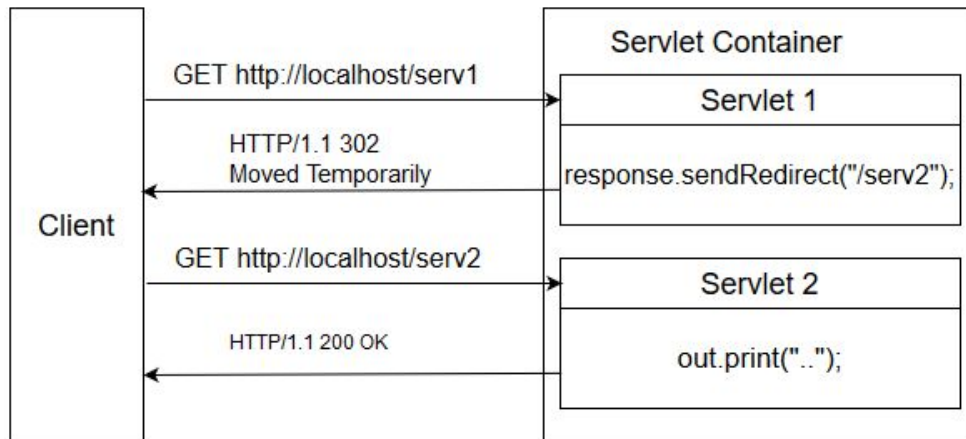
```
@WebServlet(urlPatterns = "/hello")  
public class HelloServlet extends HttpServlet {  
  
    @Override  
    protected void doGet(HttpServletRequest req,  
        HttpServletResponse resp) throws IOException  
    {  
  
        ...  
  
    }  
}
```

Forward

Метод `forward()` класа `RequestDispatcher` дозволяє перенаправити запит із сервлета на інший сервлет, html-сторінку або сторінку jsp.

```
RequestDispatcher dispatcher = getServletContext()  
    .getRequestDispatcher("/servlet2");  
dispatcher.forward(req, resp);
```

Redirect



```
resp.sendRedirect(req.getContextPath() + "/servlet2");
```

Після цього методу клієнту не має відправлятися більше ніяких даних цим сервлетом (в ідеалі – має закінчуватись виконання сервлету)

Filter

```
@WebFilter("/*")
public class MyFilter implements Filter {

    @Override
    public void init(FilterConfig filterConfig) throws ServletException {}

    @Override
    public void doFilter(
        ServletRequest request,
        ServletResponse response,
        FilterChain chain
    ) throws IOException, ServletException {

        System.out.println("!!! In filter: " + ((HttpServletRequest) request).getRequestURL());
        chain.doFilter(request, response);
    }

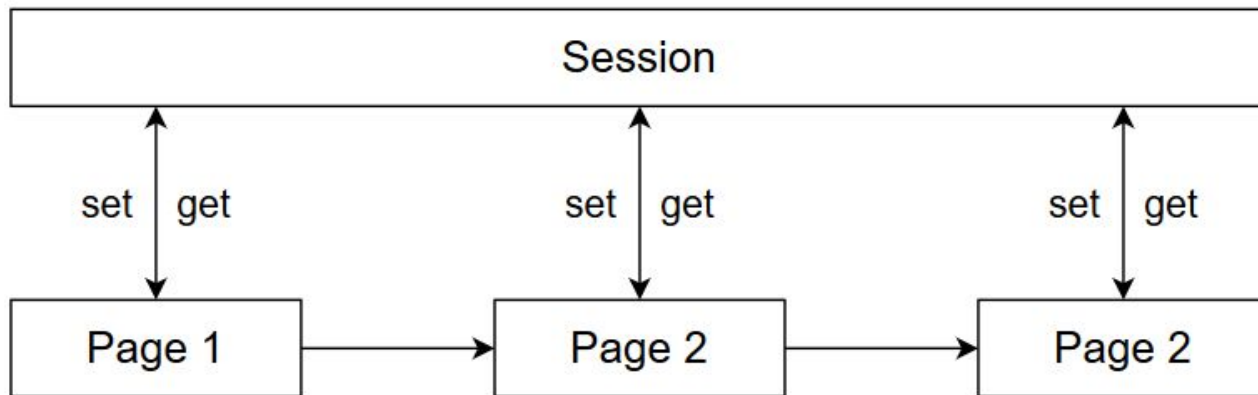
    @Override
    public void destroy() {}
}
```

Об'єкт **сесії** створюється кожен раз при отриманні запиту від нового клієнта і в результаті ідентифікує його унікальним чином.

За різними користувачами стоять окремі об'єкти сесій

Сесії “живуть” на сервері протягом заданого часу, але лиш для одного клієнтського браузера

Session



```
// кладемо об'єкт в сесію  
req.getSession().setAttribute("attr1", value);
```

```
// отримуємо об'єкт із сесії  
req.getSession().getAttribute("attr1")
```

Реалізація сесій

У випадку використання cookies автоматично формується Cookie з іменем **JSESSIONID** і значенням виду **02395C69B8FB84D3278B49F1B05F3379**, яке використовується в якості ключа в хеш-таблиці на сервері

Якщо **cookie** вимкнені, формується URL виду:
http://.../some_path;jsessionid=02395C69B8FB84D3278B49F1B05F3379

Приклади роботи з сесією

Фрагмент кода сервлета, що перевіряє правильність вводу імені і пароля

```
// Беремо параметри із запиту
String login = req.getParameter("login");
String password = req.getParameter("password");
// створюємо об'єкт User
User user = new User(login, password);
if (userDao.contains(user)){
    // Якщо такий є – розміщуємо у сесію
    req.getSession().setAttribute("user", user);
}
```

Особливості використання сесій

- Надто багато всього в сесії може створювати суттєве навантаження на сервер
- Сесії “живуть” в оперативній пам’яті одного екземпляра вашого web-додатка
- Якщо потрібно мати консистентний стан на всіх екземплярах, то можна
 - Використовувати persistence + кеш (замість сесій)
 - Серіалізувати сесії

Дані, загальні для всього застосунку

Об'єкт `ServletContext` існує в одному екземплярі для одного WEB-застосунку

В ньому можна зберігати глобальні налаштування застосунку (за допомогою методів `get/setAttribute(...)`)

Інші методи `ServletContext`:

- `getRealPath(String path)` – повертає реальний шлях до ресурсу файлової системи, що знаходиться по заданому віртуальному шляху
- `getResourceAsStream(String path)` – повертає потік байт реального ресурса файлової системи

Отримати посилання на `ServletContext` із сервлета можна, наприклад, так:

- `getServletContext()`
- `req.getSession().getServletContext()`

Jakarta Server Pages

JSP (Jakarta Server Pages) — технологія, що дозволяє веб-розробникам створювати вмісти, які мають як статичні, так і динамічні компоненти.



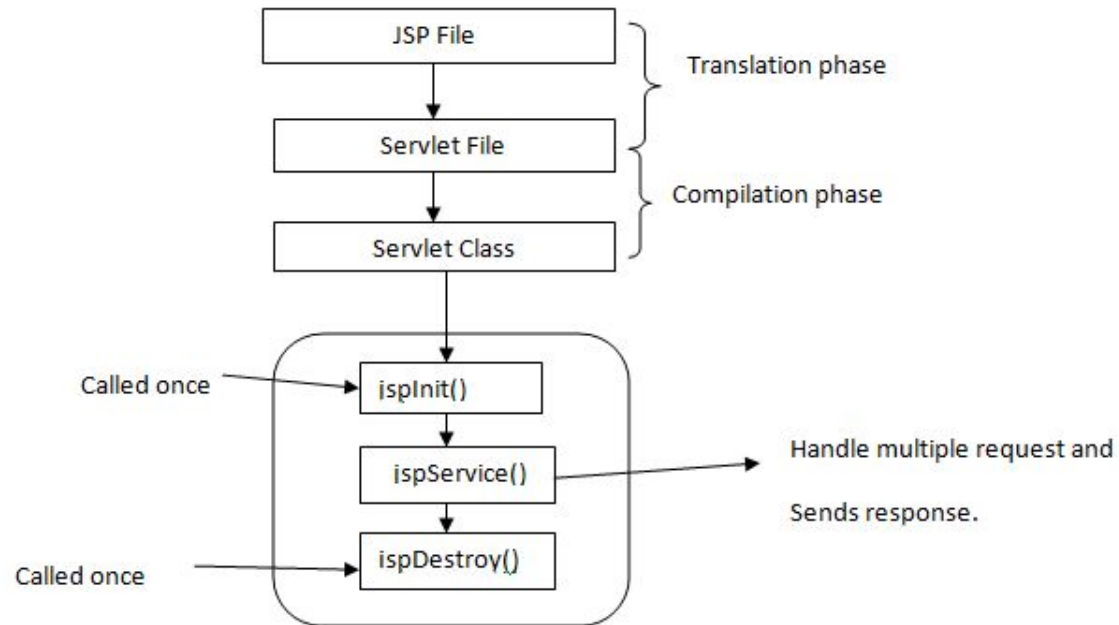
Що зручніше?

```
public class SimpleServlet extends javax.servlet.http.HttpServlet {  
    public void doGet(HttpServletRequest request,  
        HttpServletResponse response) throws IOException {  
        PrintWriter out = response.getWriter();  
        out.println("<html>");  
        out.println("<head><title>SimpleServlet</title></head>");  
        out.println("<body bgcolor=\"#ffff0f\">");  
        out.print("<p>Hello ");  
        out.print(request.getParameter("name"));  
        out.print("</p>");  
        out.println("</body></html>");  
    }  
}
```

або

```
<html>  
<head><title>SimpleServlet</title></head>  
<body bgcolor="#ffff0f">  
<p>Hello <%= request.getParameter("name") %> </p>  
</body></html>
```

JSP Life Cycle



Додавання коду в JSP-сторінках

Вирази (expressions)

```
<%=user.getName()%>
```

Скриптлети (scriptlets) – код, що додається в метод `_jspService()` сервлета

```
<% System.out.println("Hello"); %>
```

Декларації (declarations) – ділянки коду, що додаються в тіло сервлета поза його методів

```
<%! declarations %>
```

Коментарі

```
<!-- This text will not appear in the response --%>
```

Директиви

Директиви JSP повинні розміщуватись всередині
<%@ ... %>

<%@ **page** **pageEncoding**="UTF-8" %> - задання кодування сторінки

<%@ **page** **contentType**="text/html"%> - задання типу вмісту

<%@ **page** **import**="java.util.List" %> - імпорт класу

і т.д.

Вкладання файлів

Існує 2 основних способи вкладання файлів в тіло основної сторінки:

На етапі **трансляції сторінки**. В цьому випадку код вкладеної сторінки буде розміщений в кодї сервлета основної сторінки

```
<%@ include file="menu.jsp" %>
```

На етапі **виконання запитів**. В цьому випадку сторінка буде вкладатися кожен раз динамічно

```
<jsp:include page="menu.jsp" flush="true" />
```

Expression Language

Вирази EL, що мають вигляд:

`${ expression }`

де expression – текст виразу відповідний до синтаксису

Дозволяє вставляти runtime вирази на рівні:

- статичного тексту JSP сторінки(т.з. Template)
- значень атрибутів CustomTags

За своїм призначенням вирази багато в чому схожі з виразами `<%= ... %>`, але є більш зручним інструментом роботи з даними

Використання JavaBeans на JSP

Створення JavaBean-компонентів

```
<jsp:useBean id="studentBean"  
             class="package.StudentBean"  
             scope="request" />
```

Доступ до полів компонентів:

```
<%= studentBean.getName() %>
```

або

```
<jsp:getProperty name="studentBean" property="name" />
```

або

```
${studentBean.name}
```

scope бінів

Bean-компоненти можуть зберігатись в різних місцях в залежності від параметра `scope`

- **page** – значення за замовчуванням. Область видимості – поточна сторінка.
- **request** – зберігається в `HttpServletRequest`
- **session** – зберігається в `HttpSession`
- **application** – зберігається в класі `ServletContext`

```
<jsp:useBean id="user" class="my.User" scope="session">
```

Приклад Servlet -> JSP

```
public class DisplayStudentServlet extends HttpServlet {  
    @Override  
    protected void doGet(HttpServletRequest req,  
                           HttpServletResponse resp)  
        throws ServletException, IOException {  
        String id = req.getParameter("id");  
        if ("1".equals(id)) {  
            StudentBean stud = new StudentBean("1", "Vasya", 22);  
            req.setAttribute("studentBean", stud);  
            RequestDispatcher rd = getServletContext()  
                .getRequestDispatcher("/WEB-INF/studentView.jsp");  
            rd.forward(req, resp);  
        } else {  
            resp.sendError(404);  
        }  
    }  
}
```

studentView.jsp

```
<jsp:useBean id="studentBean"
              class="ua.profitsoft.ajd.lk5web.StudentBean"
              scope="request"/>

<table>
  <tr>
    <td>ID</td><td>${studentBean.id}</td>
  </tr>
  <tr>
    <td>Name</td><td><%= studentBean.getName() %></td>
  </tr>
  <tr>
    <td>Age</td>
    <td>
      <jsp:getProperty name="studentBean" property="age" />
    </td>
  </tr>
</table>
```

JavaServer Pages Standard Tag Library (JSTL)

Бібліотека тегів JSTL зберігає набір основної функціональності, загальної для багатьох JSP застосунків

- умовна обробка,
- створення циклів
- підтримка інтернаціоналізації
- та інше

Підключення в pom.xml:

```
<dependency>  
  <groupId>javax.servlet</groupId>  
  <artifactId>jstl</artifactId>  
  <version>1.2</version>  
</dependency>
```

Цикли в JSTL

```
<%@taglib uri="http://java.sun.com/jsp/jstl/core"
          prefix="c" %>
```

```
<table>
```

```
  <thead><td>ID</td><td>Name</td><td>Age</td></thead>
```

```
  <c:forEach var="student" items="{studentList}">
```

```
    <tr>
```

```
      <td>${student.id}</td>
```

```
      <td>${student.name}</td>
```

```
      <td>${student.age}</td>
```

```
    </tr>
```

```
  </c:forEach>
```

```
</table>
```


Постачання і запуск web-застосунку (без IDEA)

складає war-файл

```
mvn package
```

Кладемо `myapp.war` в `tomcat/webapps`
і запускаємо `tomcat`

```
startup.bat
```